DSP in VLSI

Homework 3

Interpolator

Student ID

M11407439

Name

MING-HONG, LIN

Development Environment

HDL	SystemVerilog
Simulation Tools	Synopsys VCS and Verdi
Synthesis Tool	Synopsys DC Compiler
Process	TSMC 16nm (ADFP)
Verification	Python

Due Date: 2026/04/15

Contents

1 Step 1 — Interpolator Function Verification **2**
1.1 Result 1 - Linear vs Second-Order Polynomial Interpolator 2

2 Step 2 — Interpolator Function Verification **4**
2.1 Result 2 - Linear vs Piecewise Parabolic Interpolator 4

3 Step 3 — Bit-True Model (BF16) **5**
3.1 Result 3 - BF16 Operation Results 5

4 Step 4 — Performance Degradation by Finite Precision **6**
4.1 Result 4 - Quantization Error Analysis 6

5 Step 5 — Hardware Reduction **7**
5.1 Result 5 - Original Block Diagram 7
5.2 Result 5 - Hardware Sharing Approaches 8

6 Step 6 — Implementation **9**
6.1 Result 6 - Behavior Simulation & Error Analysis 9
6.2 Result 7 - Timing Report and Constraint 10
6.3 Result 8 - Post-Synthesis Simulation 12
6.4 Result 8 - BF16 Bit-True Model vs. Verilog Output 13

Step 1 — Interpolator Function Verification

1.1 Result 1 - Linear vs Second-Order Polynomial Interpolator

Assume that

$$x_1[m] = \cos\left(2\pi\left(\frac{mT_s}{10T_s} + \frac{\phi}{2}\right)\right) + j\sin\left(2\pi\left(\frac{mT_s}{10T_s} + \frac{\phi}{2}\right)\right),$$

where $\phi = \mathrm{mod}(d, 2)$ and d is the last digit of your student ID. The sampling frequency is 10 times the frequency of the sinusoidal wave. Please use **linear interpolator** and **second-order polynomial interpolator** to interpolate the sampled waveform in the region of $10 \le m \le 20$ with $\mu = 0, \frac{1}{8}, \frac{2}{8}, \ldots \frac{7}{8}$. Note that you have $x_1[9]$ when you want to interpolate $x_1[10.125]$. Show the **real-part** and **imaginary-part** output waveform $\hat{x}_1[m+\mu]$ after interpolation for both methods. In addition, draw the absolute error $|x_1[m+\mu] - \hat{x}_1[m+\mu]|$ in the region of $10 \le m \le 20$ with $\mu = 0, \frac{1}{8}, \frac{2}{8}, \ldots \frac{7}{8}$. (30%)

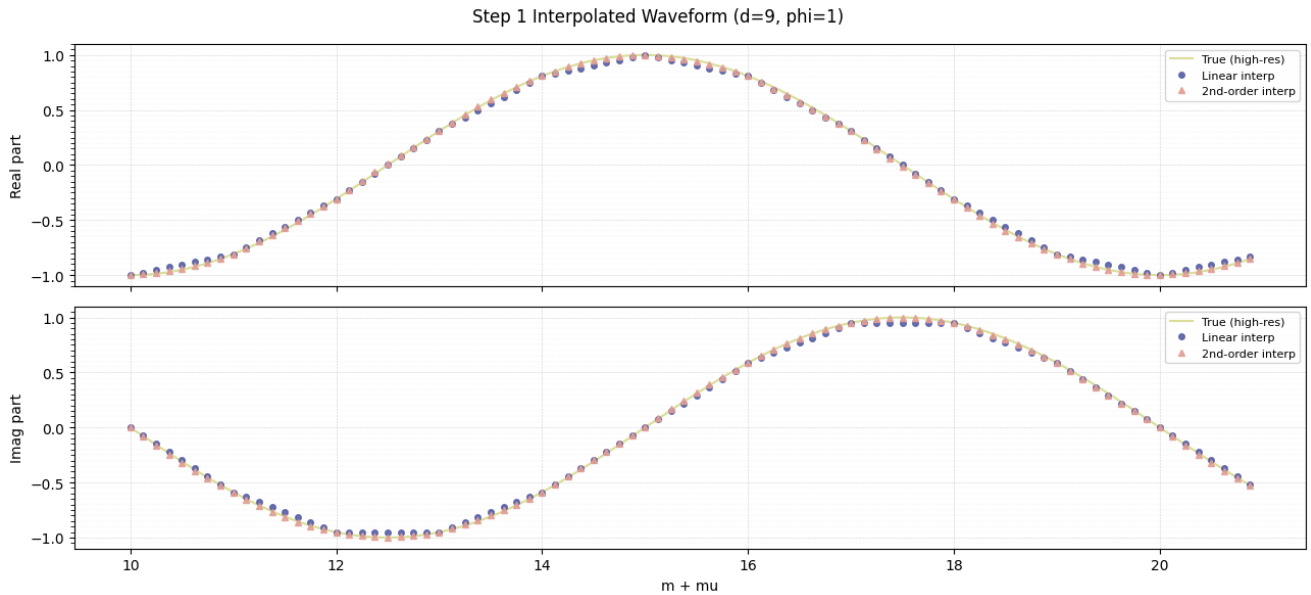

Figure 1.1: Waveform of Linear vs Second-Order Polynomial Interpolator

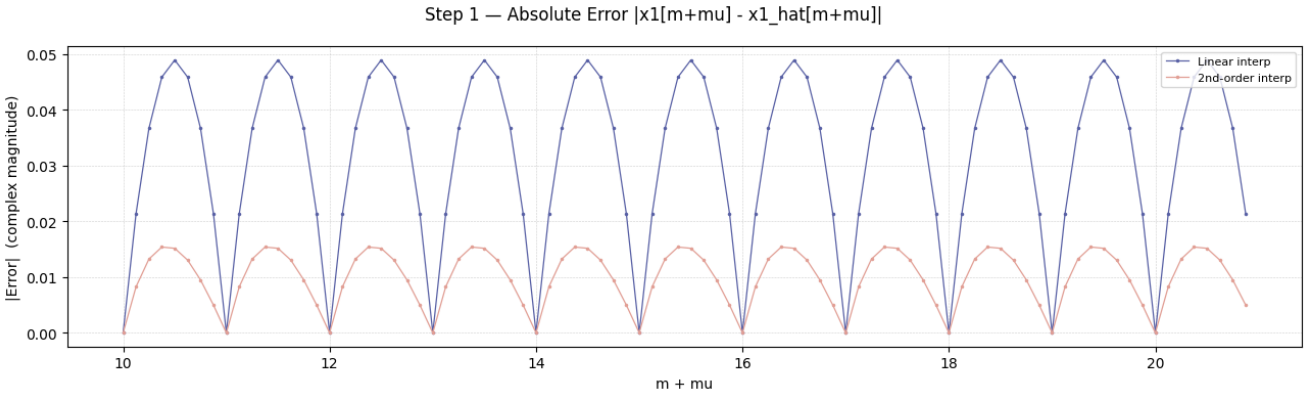

Figure 1.2: Absolute Error Comparison of Linear vs Second-Order Polynomial Interpolator

Step 2 — Interpolator Function Verification

2.1 Result 2 - Linear vs Piecewise Parabolic Interpolator

Assume that

$$x_2[m] = 2^{-20}\left[\cos\left(2\pi\left(\frac{mT_s}{4T_s} + \frac{\phi}{3}\right)\right) + j\sin\left(2\pi\left(\frac{mT_s}{4T_s} + \frac{\phi}{3}\right)\right)\right],$$

where ϕ is the same parameter according to your student ID. The sampling frequency is 4 times the frequency of the sinusoidal wave. Please use **linear interpolator** and **piecewise parabolic interpolator** to interpolate the sampled waveform in the region of $4 \le m \le 8$ with $\mu = 0, \frac{1}{9}, \frac{2}{9}, \ldots \frac{8}{9}$. Show the **real-part** and **imaginary-part** output waveform $\hat{x}_2[m+\mu]$ after interpolation. In addition, draw the absolute error $|x_2[m+\mu] - \hat{x}_2[m+\mu]|$ in the region of $4 \le m \le 8$ with$\mu = 0, \frac{1}{8}, \frac{2}{8}, \ldots \frac{7}{8}$. (30%)

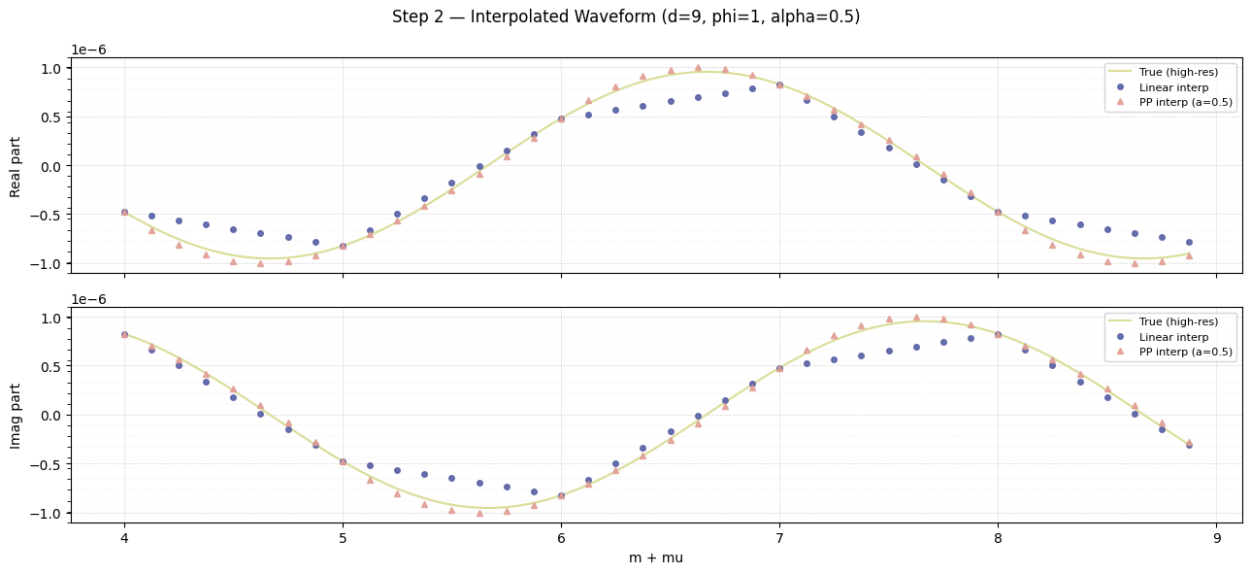

Figure 2.1: Waveform of Linear vs Piecewise Parabolic Interpolator

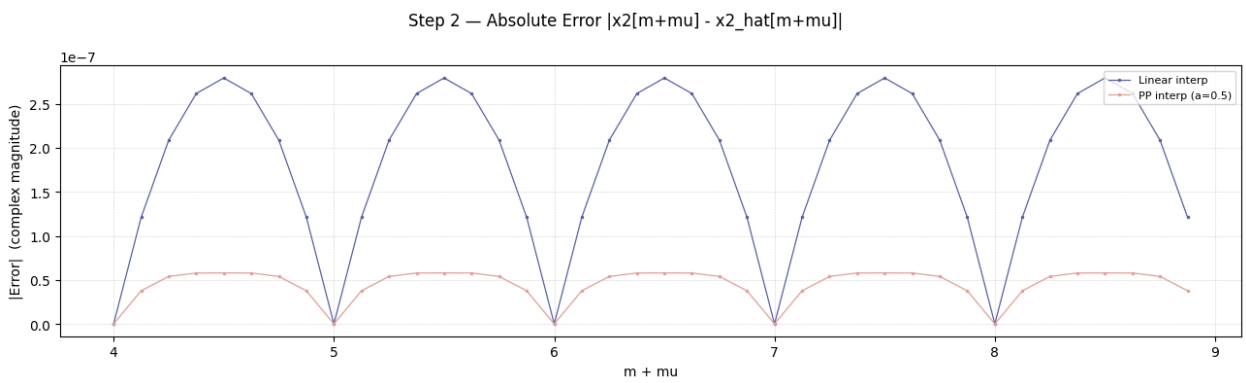

Figure 2.2: Absolute Error Comparison of Linear vs Piecewise Parabolic Interpolator

Step 3 — Bit-True Model (BF16)

3.1 Result 3 - BF16 Operation Results

Please use **BF16 format** to realize addition and multiplication. Write the bit-true model (simulated program in Python) to realize BF16 additions and multiplication. For simplicity, NaN, infinity, and subnormal numbers are not handled — if the magnitude is smaller than the smallest normal number, use **truncation** to make it become 0. Truncation is also used to avoid renormalization. Show the results calculated by your BF16 bit-true model for the following operands. Express results both in **decimal** and **binary** representation (S = sign bit, E = exponent field, F = fraction field, all in binary). **(20%)**

	Operand 1	**Operand 2**	**Operation**
(a)	S: 1 E: `1000_0011` F: `0000_000`	S: 0 E: `1000_1011` F: `0000_011`	Addition
(b)	S: 1 E: `0000_0001` F: `0000_011`	S: 0 E: `0000_0001` F: `1111_010`	Addition
(c)	S: 1 E: `0000_0010` F: `1100_000`	S: 0 E: `0111_1100` F: `0000_110`	Multiplication
(d)	S: 0 E: `0110_0011` F: `1011_110`	S: 0 E: `1001_0011` F: `0101_000`	Multiplication

Table 3.1: BF16 operands and operations.

The following are the four option results obtained from a BF16 Bit-True Model implemented in Python.

	Operation	**Decimal**	**S**	**E & F (binary)**
(a)	Addition	4192	0	E: `1000_1011`, F: `0000_011`
(b)	Addition	0	0	E: `0000_0000`, F: `0000_000`
(c)	Multiplication	0	0	E: `0000_0000`, F: `0000_000`
(d)	Multiplication	0.0088501	0	E: `0111_1000`, F: `0010_001`

Table 3.2: BF16 operation results.

Step 4 — Performance Degradation by Finite Precision

4.1 Result 4 - Quantization Error Analysis

Please analyze the error of the **second-order interpolator** if its arithmetic operations are changed to BF16 by using the bit-true model. Draw the figures indicating the difference of interpolated results caused by data format for outputs $\hat{x}_1[m+\mu]$ in the region of $10 \le m \le 20$ with $\mu = 0, \frac{1}{8}, \frac{2}{8}, \ldots \frac{7}{8}$. **(10%)**

As shown in the figure below, switching the arithmetic from double precision to BF16 introduces a noticeable quantisation error due to the reduced mantissa width (7 bits) of the BF16 format. The peak absolute errors observed over the interpolation range $10 \le m \le 20$ are summarised below:

Component	**Max Absolute Error**
Real part	1.5959×10^{-2}
Imaginary part	1.6048×10^{-2}

Table 4.1: Maximum absolute error introduced by BF16 arithmetic in the second-order interpolator.

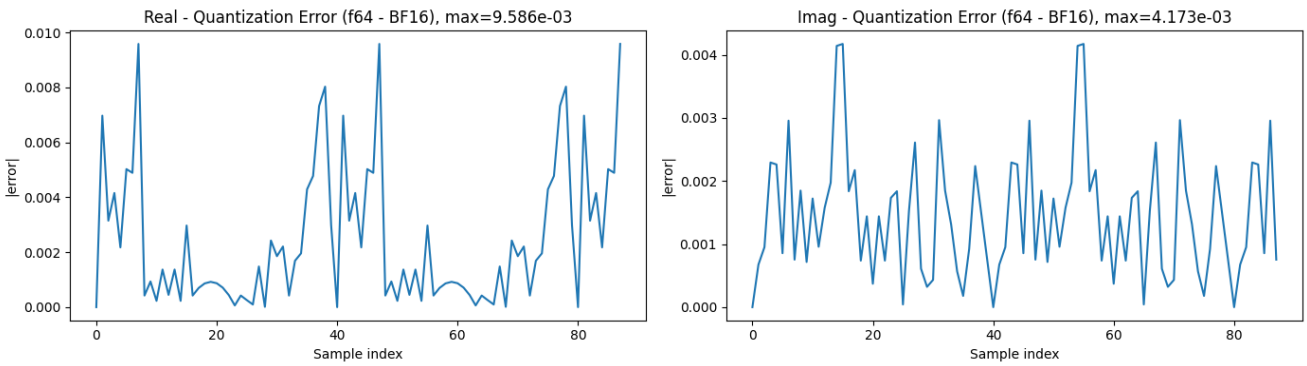

Figure 4.1: Interpolation Error Caused by BF16 Finite Precision (Second-Order Interpolator)

Step 5 — Hardware Reduction

5.1 Result 5 - Original Block Diagram

Draw the complete block diagram of your first version implementation according to Fig. 4. (5%)

The second-order polynomial interpolator in Farrow structure decomposes the interpolation into three fixed sub-filters v_0, v_1, and v_2, whose outputs are combined as

$$\hat{x}[m+\mu] = v_0 + \mu\, v_1 + \mu^2\, v_2.$$

The coefficients of each sub-filter are derived from three consecutive input samples $X[m]$, $X[m+1]$, and $X[m+2]$ as summarised in Table 5.1.

Sub-filter	$X[m]$	$X[m+1]$	$X[m+2]$
v_2	$+\frac{1}{2}$	-1	$+\frac{1}{2}$
v_1	$-\frac{3}{2}$	$+2$	$-\frac{1}{2}$
v_0	$+1$	0	0

Table 5.1: Farrow sub-filter coefficients for the second-order polynomial interpolator.

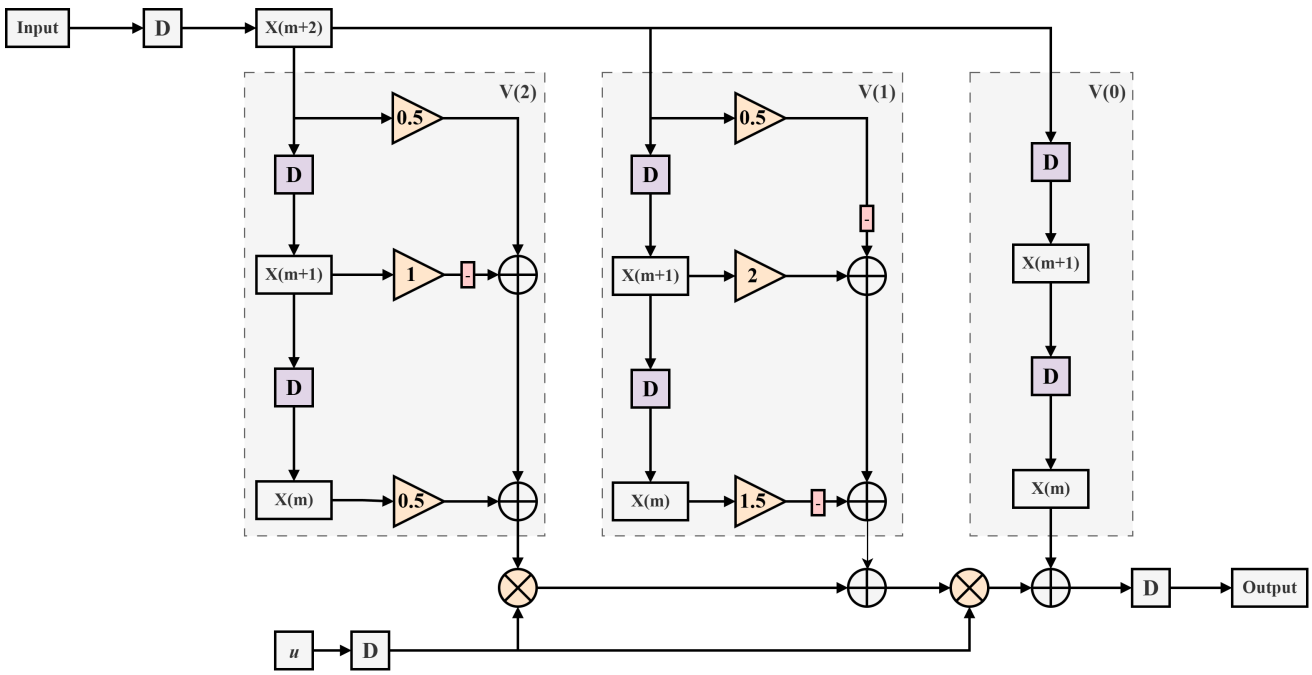

Figure 5.1: Original Block Diagram (Farrow Structure)

5.2 Result 5 - Hardware Sharing Approaches

Show your hardware sharing version (The precision of the datapath must be BF16 and can not be changed.) (10%) Draw the block diagram after your improvement. (5%) Indicate the critical path (5%)

Three hardware optimizations were applied to reduce resource usage and improve operating frequency.

1. **Shared delay-line (single DFF chain) :** The original structure uses three independent delay lines, one per sub-filter branch. Since all three branches consume the same input samples $X[m]$, $X[m+1]$, and $X[m+2]$, a single shared shift-register chain suffices, eliminating two redundant sets of D flip-flops.

2. **Multiplier-free scaling :** In BF16 format, multiplication by 0.5 **reduces to decrementing the exponent field by one and no multiplier is needed.** Accordingly, all $\times 0.5$ operations are replaced by a one-bit exponent adjustment. The coefficient $\times 2$ is likewise converted to an equivalent **exponent add one**, and $\times 1.5$ is decomposed into $A + 0.5A$, so that the entire datapath requires no general-purpose multipliers, only Mu datapath needs.

3. **Two-stage pipeline insertion :** To increase the maximum operating frequency, two pipeline registers are inserted across the critical path, balancing the combinational logic depth between stages.

As we can see, the critical path is **the purple path** in Figure 5.2. It starts from **Input Register** and ends at **Pipeline Stage 1 Register** that includes one multiplier, one $\times 0.5$ unit and two adder.

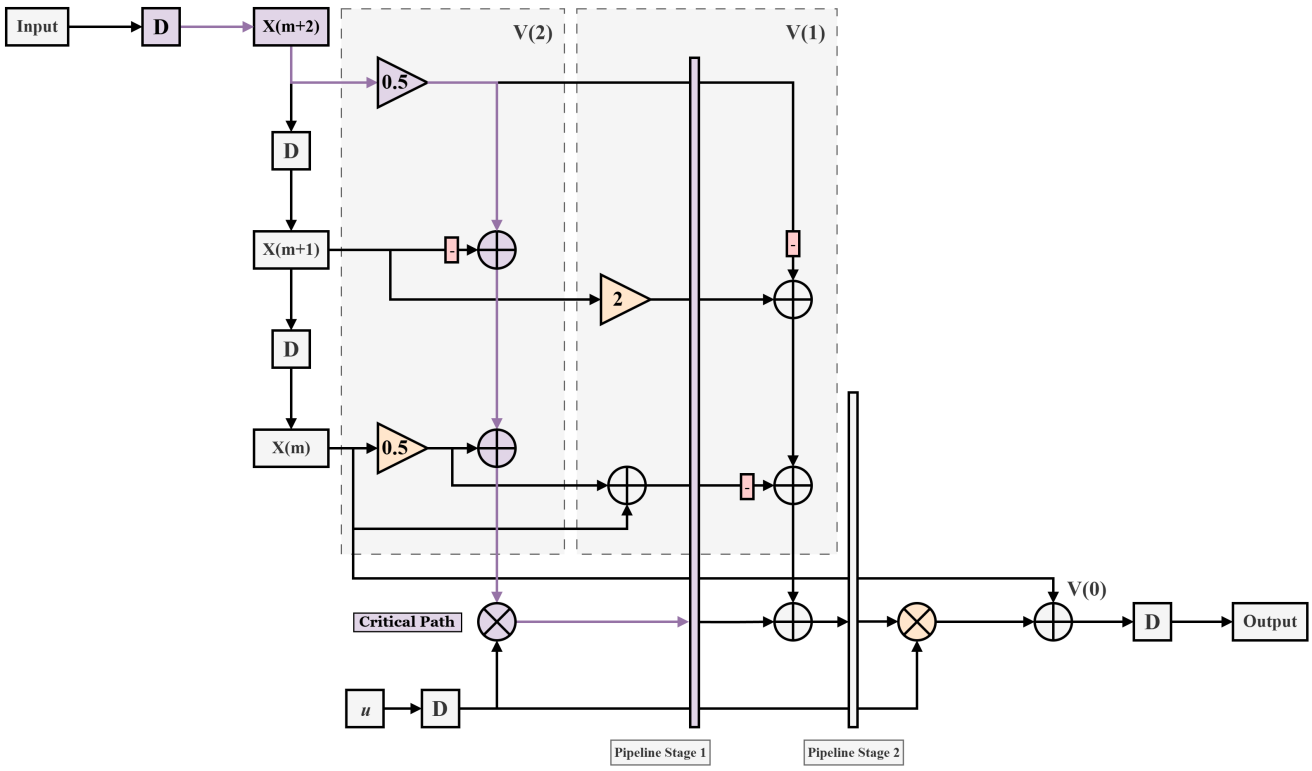

Figure 5.2: Block Diagram with Hardware Sharing Approaches

Step 6 — Implementation

6.1 Result 6 - Behavior Simulation & Error Analysis

Show the timing diagram of behavior simulation for the **first 8** and **last 8** ($\mu = 0, \frac{1}{8}, \ldots \frac{7}{8}$) interpolated outputs of $\hat{x}_1[m + \mu]$ in the region of $10 \leq m \leq 20$. Show the **real-part** and **imaginary-part** error of hardware implementation results compared to the double-precision results. **(30%)**

Due to the two-stage pipeline architecture and the delayed feedback loops (DFF) at the input and output stages, the design introduces a latency of 4 clock cycles between input and output. As shown in Figure 6.2, the output results are fully presented 4 clock cycles after the input signal Mu stops to enter the system.

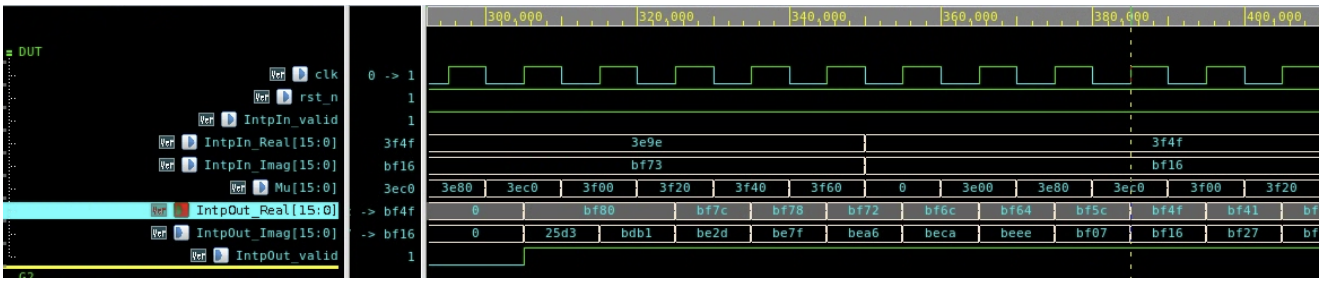

Figure 6.1: Timing diagram of behavior simulation for the first 8

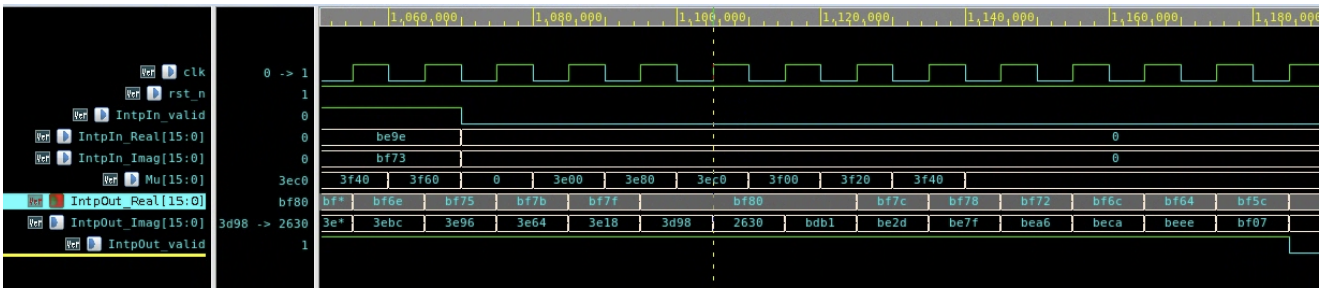

Figure 6.2: Timing diagram of behavior simulation for the last 8

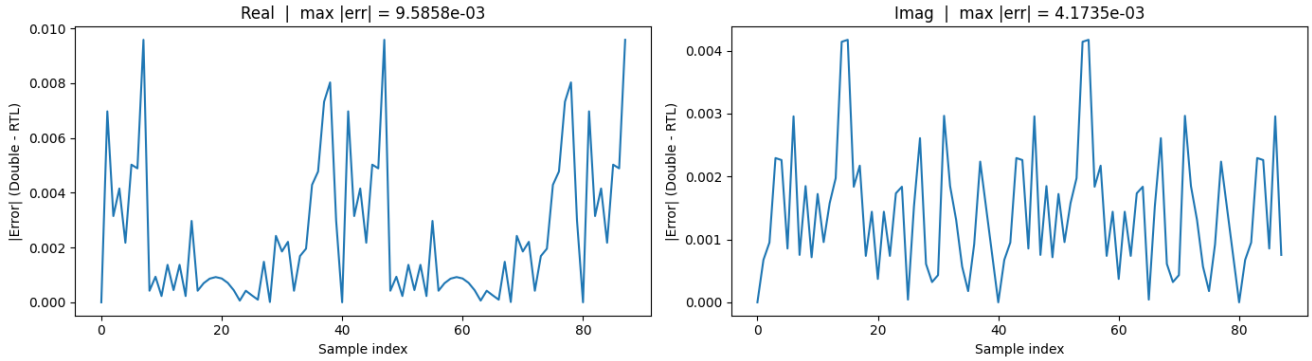

Figure 6.3: Error Analysis of hardware results compared to the double-precision results

6.2 Result 7 - Timing Report and Constraint

how your **timing report** to verify the critical path of the block diagram in Step 5. **(10%)** List your **timing constraint** for post-synthesis simulation. **(5%)**

Parameter	Value (ns)
Clock Period	0.8
Clock Uncertainty	0.1
Clock Latency	0.1
Input / Output Maximum Delay	0.5

Table 6.1: Timing constraint settings provided by TA

According to the Timing Report, the critical path starts from **IntpIn_Reg** Register and ends at **uV2_TEMP** Register, which is consistent with the purple path depicted in Section 5.2.

```
Operating Conditions: ff0p88v125c   Library: N16ADFP_StdCellff0p88v125c_ccs
Wire Load Model Mode: segmented

  Startpoint: Imag_Part/IntpIn_Reg_reg[0][11]
              (rising edge-triggered flip-flop clocked by clk)
  Endpoint: Imag_Part/uV2_TEMP_reg[6]
            (rising edge-triggered flip-flop clocked by clk)
  Path Group: clk
  Path Type: max

  Des/Clust/Port     Wire Load Model       Library
  ------------------------------------------------
  Interpolator       ZeroWireload          N16ADFP_StdCellff0p88v125c_ccs
  Common_Logic_0     ZeroWireload          N16ADFP_StdCellff0p88v125c_ccs

  Point                                                    Incr       Path
  -----------------------------------------------------------------------
  clock clk (rise edge)                                    0.00       0.00
  clock network delay (ideal)                              0.50       0.50
  Imag_Part/IntpIn_Reg_reg[0][11]/CP (DFCNQD2BWP16P90LVT)
                                                           0.00       0.50 r
  Imag_Part/IntpIn_Reg_reg[0][11]/Q (DFCNQD2BWP16P90LVT)
                                                           0.03       0.53 f
```

Figure 6.4: Report_Timing 1

As demonstrated in the report, the design achieves a **maximum operating frequency of 1.25 GHz** without any setup and hold time violations.

```
Imag_Part/U990/Z (AO21D2BWP16P90)                        0.01       1.09 r
Imag_Part/U3184/Z (XOR3D4BWP16P90LVT)                    0.03       1.12 f
Imag_Part/U4139/ZN (NR2D2BWP16P90LVT)                    0.01       1.13 r
Imag_Part/U308/ZN (OAI21D1BWP16P90)                      0.02       1.14 f
Imag_Part/U4221/ZN (AOI21D1BWP16P90)                     0.01       1.15 r
Imag_Part/U531/ZN (OAI21D1BWP16P90LVT)                   0.01       1.16 f
Imag_Part/U1644/ZN (XNR2D2BWP16P90LVT)                   0.01       1.18 r
Imag_Part/U3196/ZN (AOI22D1BWP16P90LVT)                  0.01       1.18 f
Imag_Part/U3192/ZN (ND2D1BWP16P90LVT)                    0.01       1.19 r
Imag_Part/uV2_TEMP_reg[6]/D (DFCNQD1BWP20P90LVT)         0.00       1.19 r
data arrival time                                                   1.19

clock clk (rise edge)                                    0.80       0.80
clock network delay (ideal)                              0.50       1.30
clock uncertainty                                       -0.10       1.20
Imag_Part/uV2_TEMP_reg[6]/CP (DFCNQD1BWP20P90LVT)        0.00       1.20 r
library setup time                                      -0.01       1.19
data required time                                                  1.19
-------------------------------------------------------------------------
data required time                                                  1.19
data arrival time                                                  -1.19
-------------------------------------------------------------------------
slack (MET)                                                         0.00
```

Figure 6.5: Report_Timing 2

6.3 Result 8 - Post-Synthesis Simulation

Show the timing diagram of **post-synthesis simulation** for the first 8 and last 8 interpolated outputs of $\hat{x}_1[m+\mu]$ in the region of $10 \leq m \leq 20$. Show the **real-part** and **imaginary-part** error of post-synthesis hardware outputs compared to the double-precision results. **(20%)**

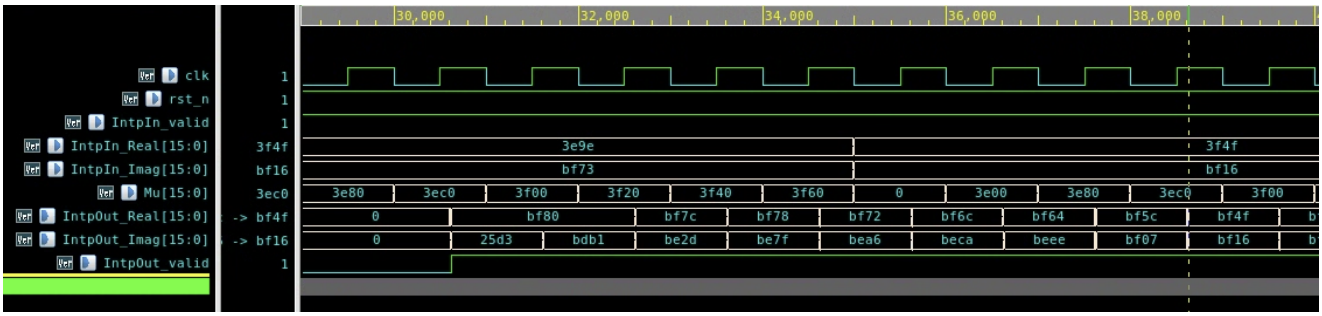

Figure 6.6: Timing diagram of post-synthesis simulation for the first 8

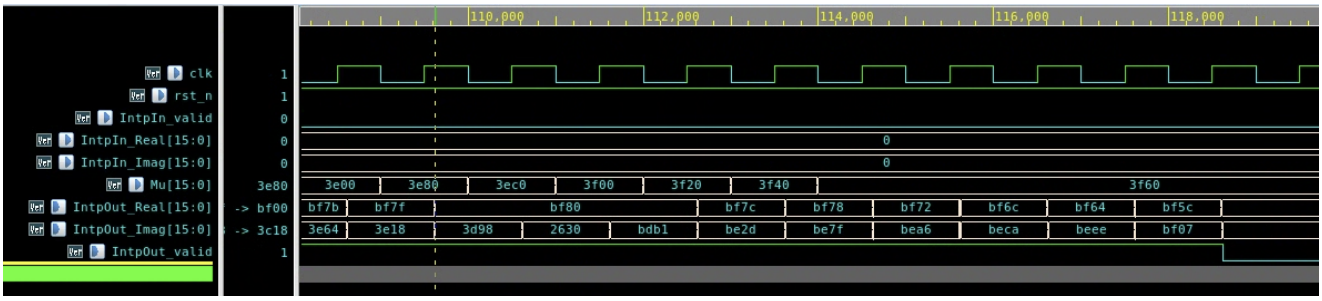

Figure 6.7: Timing diagram of post-synthesis simulation for the last 8

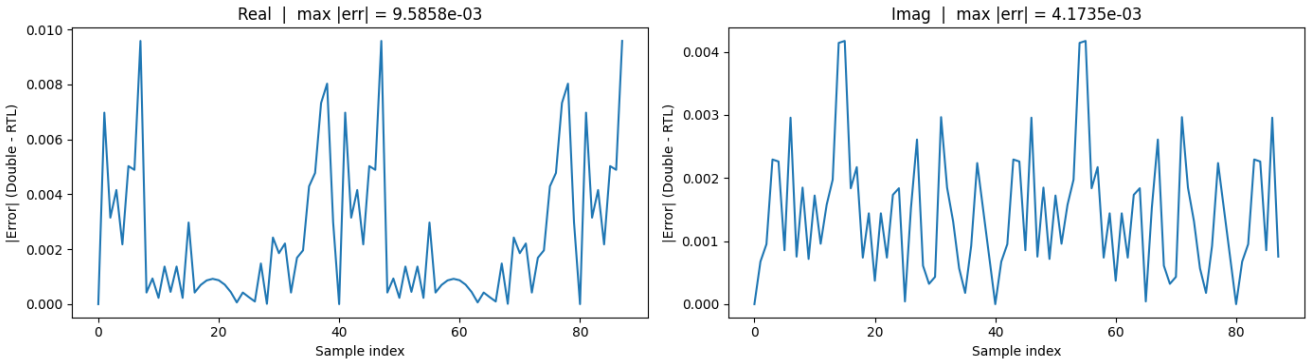

Figure 6.8: Error Analysis of hardware results compared to the double-precision results (Post-Synthesis)

6.4 Result 8 - BF16 Bit-True Model vs. Verilog Output

Construct the **BF16 bit-true model** following the architecture after hardware improvement. Depict the **real-part** and **imaginary-part** errors of interpolated outputs $\hat{x}_1[m+\mu]$ between the Verilog outputs and the bit-true model in the region of $10 \leq m \leq 20$ with $\mu = 0, \frac{1}{8}, \frac{2}{8}, \ldots \frac{7}{8}$. **(20%)**

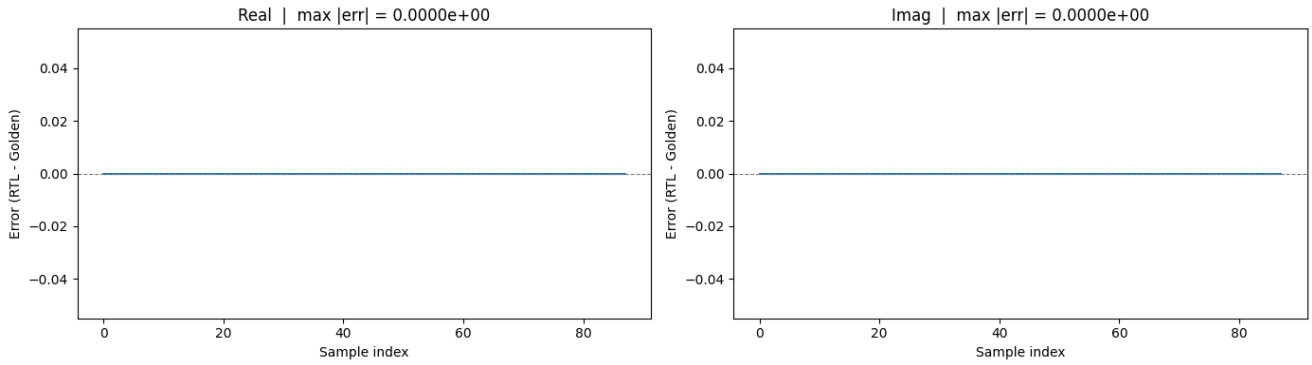

Figure 6.9: BF16 Bit-True Model vs. Verilog Output